Dialback System Design

Oct 31, 2026 · @Joshua Noji

Summary

What we're building. Dialback automates the callback that AP teams are told to make before paying a vendor's new bank account. When a vendor's bank details change in Ramp, Dialback holds payments to that vendor, picks a phone number it can prove belongs to the vendor, and has an AI agent make a disclosed call. Code decides the outcome, and a person who didn't make the change releases the payment. By Nov 10 it runs end to end against a Ramp-shaped mock (and Ramp's sandbox if we get access), with live calls between our own phones, a synthetic company of 500 vendors, and an attack lab that replays about 300 scams and 60 legitimate-but-odd changes.

The architecture in one paragraph. A Next.js dashboard on Vercel talks to Python services on Railway: a core service that runs each case and talks to Ramp, a voice service that runs live calls through Twilio ConversationRelay, and a Ramp-shaped mock. Supabase Postgres sits in the middle as the referee. SQL functions enforce the state machine, the four-eyes release and a hash-chained evidence log, so a bug anywhere else can't move money. The Trust Ledger and the decision engine are Python packages inside the core service, and the attack lab imports those same packages and runs them on a simulated clock. Models do two jobs only, reading documents and holding the conversation; code picks the number and decides.

Who owns what. Abhi owns the money path: Ramp, cases, decisions, release and the evidence log. Inesh owns the evidence: the synthetic company, the Trust Ledger and number selection. Sami owns the call and the adversary: the voice agent and the attack lab. The three slices meet only at contracts frozen on day 1, so each person can build in parallel with coding agents.

Key dates. Walking skeleton on Nov 1, go/no-go on the live call on Nov 4, code freeze on Nov 8, submission on Nov 10, and the live final at Ramp's San Francisco office on Nov 19.

The document follows standard system-design practice: a C4 container view, a sequence diagram, a state machine with invariants, architecture decision records, a STRIDE and OWASP threat model, and a test plan with CI gates.

What Dialback does

Dialback holds payments to any vendor whose bank details change. It releases them only after the real vendor confirms the change, on a phone number Dialback can prove belongs to them. It runs beside Ramp Bill Pay and acts through Ramp's API.

Who is involved

Who	Their part
AP clerk	Enters or receives the bank-detail change. No longer makes the call.
Approver (controller or AP manager)	Reviews the evidence, then releases or rejects. Must be a different person from whoever entered the change.
Vendor contact	Answers Dialback's call and confirms or denies the change.
Fraudster	Sends the fake request. May plant phone numbers, answer calls or hide instructions in documents.
Ramp	Source of vendors, bank-detail changes and bills, and the place the payment hold is enforced.
Builder (us)	Changes scripts and rules, and watches the attack lab's scorecard.

The workflow

1. **Detect.** A vendor's bank details change in Ramp, or a change-request email arrives.
2. **Hold.** Dialback puts payments to that vendor on hold in Ramp.
3. **Choose a number.** The Trust Ledger picks a phone number backed by two independent sources that predate the request. It rejects any number first seen in the change request's email thread, or one digit off a known number. If no number qualifies, the case goes to a person and no call is made.
4. **Call.** An AI voice agent calls that number. It says it's an AI on a recorded line, asks whether the vendor requested the change, and asks for the new account's last four digits.
5. **Decide.** Code compares the answers with the request and returns Verified, Not verified or Needs review.

6. **Release.** A different person reviews the evidence and releases or rejects. Only then does the hold lift.
7. **Record.** Every step lands in an append-only evidence log.
8. **Test.** Whenever a script, prompt, rule or ledger setting changes, the attack lab replays about 300 scams and 60 legitimate-but-odd changes, and updates the scorecard.

Rules the code must enforce

These are the system's invariants. Every component is designed around them, and the tests check each one.

#	Rule	Why
R1	No payment goes to changed bank details while a case is open	The product's whole purpose
R2	Never call a number first seen in the change request or its thread	That number belongs to the fraudster
R3	The AI collects answers; a deterministic function decides	Predictable, testable, immune to sweet talk
R4	The voice agent never says the new bank details or what it is checking	A fraudster on the line learns nothing
R5	The releaser is a person, and not the person who entered the change	Segregation of duties
R6	Every state change is written to an append-only, hash-chained log	Evidence auditors and insurers can trust
R7	Errors, timeouts and unclear answers fail closed: the hold stays and a person reviews	A bug never releases money
R8	Every call discloses AI and recording in its first sentence	Law and trust
R9	Text from documents, emails and calls is data, never instructions	Prompt-injection defense

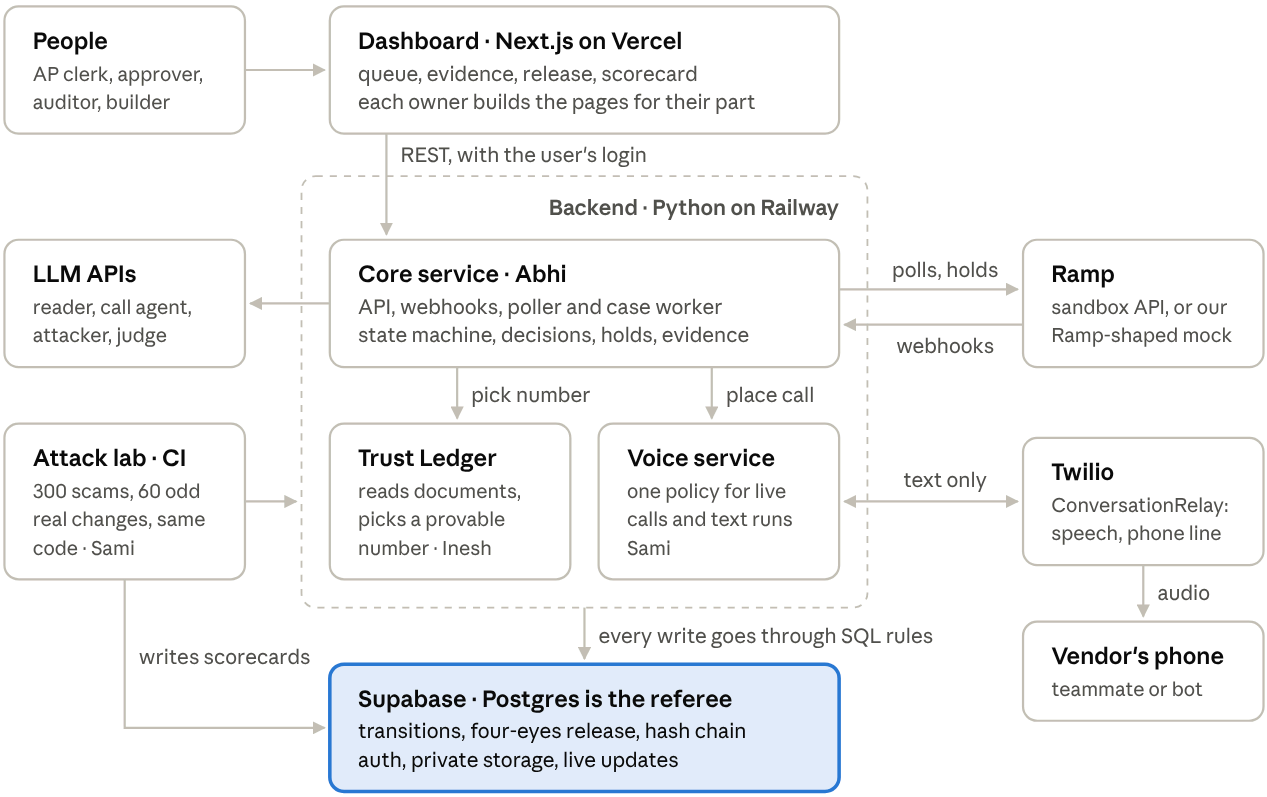
Scope for the Nov 10 submission

- **In:** the full workflow above against a Ramp-shaped mock (and Ramp's sandbox if we get access), a 500-vendor synthetic company, live AI calls between our own phones, the attack lab and scorecard, and the dashboard.
- **Out:** real vendors, production Ramp data, recovering money already sent, insurance packets, staff phishing drills, warning other Ramp customers. These are noted as future plug-ins in the last section.
- **Assumption:** student access to Ramp's API sandbox is not yet confirmed, so every Ramp call goes through one adapter that can point at the sandbox or at our mock.

Architecture overview

Dialback is a Next.js dashboard on Vercel, three Python parts on Railway, and Supabase Postgres in the middle as the referee. The rules that protect money (the state machine, four-eyes release and the evidence chain) live in SQL functions, so they hold even if application code has a bug.

Postgres is the referee: every component changes a case only through its SQL rules



Arrows show who starts each exchange. The dashboard never writes to the database directly.

Container view (C4) · 6 parts we build, 4 outside systems

The Trust Ledger and the decision engine are Python libraries that run inside the core service, which means the attack lab can import and test the exact same code.

Part	Runs on	Owner	Talks to	How
Dashboard	Next.js on Vercel	All three, one area each	Core service; Supabase Realtime for live updates	REST with the signed-in user's token; reads only
Core service (API + worker)	FastAPI on Railway, one always-on container	Abhi	Ramp, Supabase, Trust Ledger, voice service	Signed webhooks in; polling, holds and releases out; SQL functions for every state change

Part	Runs on	Owner	Talks to	How
Trust Ledger	Python package inside the core service	Inesh	LLM API, Supabase `ledger` schema	Function calls; the extractor's output must match a JSON schema
Voice service	FastAPI WebSocket server on Railway	Sami	Twilio, LLM API, Supabase	Twilio sends the caller's words as text; we send back text to speak
Attack lab	Python CLI in CI and on laptops	Sami	Ramp mock, core service, text phone line	Imports the same Python packages; writes scorecards to Supabase
Ramp mock	Small FastAPI service on Railway	Abhi	Core service	Same paths and field names as Ramp's API, with signed webhooks

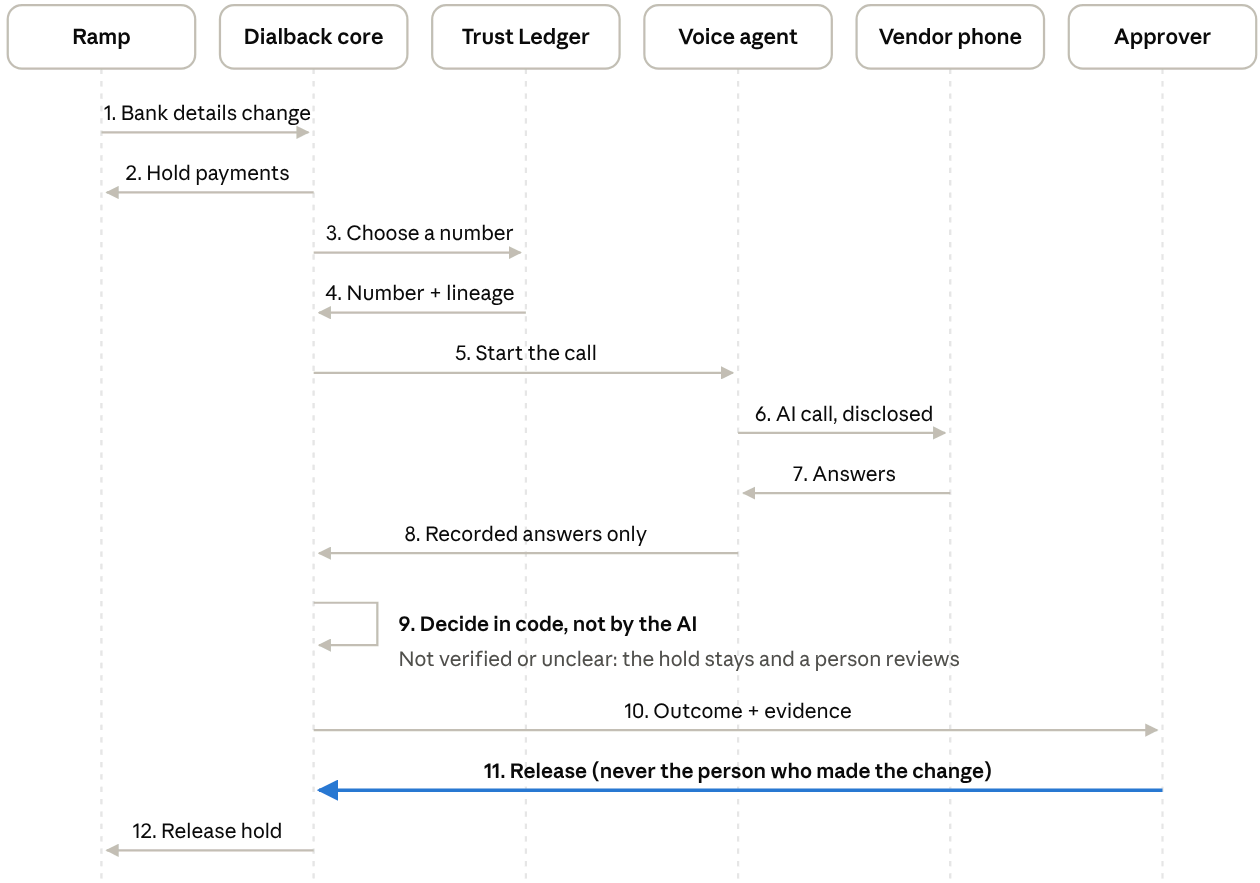
Why this shape

- **Railway, not only Vercel.** The live call needs a WebSocket that stays open for minutes, and Vercel's WebSockets are in beta and close at 300 seconds on the Hobby plan.
- **Python for the backend.** Document parsing, eval tools and Ramp's own SDK are Python-first, and one language lets the lab run the real pipeline in-process.
- **No workflow engine for now.** Postgres tables for timers and outgoing commands, run by the core worker, are enough at this size. Temporal is the production path; Ramp runs it internally.

The main flow, step by step

Every bank change becomes a case, and a case ends only with a person's release or rejection. The AI's two jobs are reading documents and holding the conversation; choosing the number and deciding the outcome are code.

Code decides, a second person releases, and only then does the hold lift



Every step is also written to the hash-chained evidence log.

Main flow · one bank change, 12 steps

The hold goes on before anything else happens (step 2), so a slow call, a crash or a confused answer can only delay a payment, never send one.

Data model

One Postgres database holds everything, and every table has exactly one owning workstream, so three people and their coding agents never edit the same tables. Bank account numbers are never stored in full: only the last four digits and a keyed hash, which is enough to compare without being able to leak.

Core tables

Table	What a row is	Key fields	Owner
`vendors`	A vendor, mirrored from Ramp	`ramp_vendor_id`, `legal_name`, `tin_hash`	Abhi
`bank_accounts`	One bank account a vendor has used	`vendor_id`, `routing_number`, `account_last4`, `account_fingerprint` (keyed hash), `first_seen_at`	Abhi
`bank_change_events`	One detected change, raw and deduplicated	`vendor_id`, `old_account_id`, `new_account_id`, `detected_via` (webhook, poll, email), `source_event_id` (unique), `changed_by`	Abhi
`cases`	The unit of work: one change being verified	`change_event_id`, `state`, `outcome`, `entered_by`, `policy_version`, `version` (for safe concurrent updates)	Abhi
`holds`	A hold placed in Ramp	`case_id`, `ramp_ref`, `placed_at`, `released_at`, `original_schedule`	Abhi
`releases`	A person's release or rejection	`case_id`, `actor_id`, `action`, `reason`; a database check rejects `actor_id = cases.entered_by`	Abhi
`documents`	One source file: invoice, contract, email, W-9, listing	`vendor_id`, `kind`, `sha256`, `custody_channel` (email, contract upload, portal, listing, paper), `received_at` (our clock), `claimed_date` (shown only, never trusted), `sender`, `thread_id`, `storage_path`	Inesh

Table	What a row is	Key fields	Owner
`observations`	"Value X was seen in document D on date T"	`document_id`, `vendor_id`, `kind` (phone, bank account, email, domain), `value_normalized`, `role`, `quote`, `visibility`, `source_type`, `seen_at` (the document's `received_at`), `extractor_version`	Inesh
`contact_points`	One phone number per vendor, with its lineage summary	`vendor_id`, `e164`, `first_seen_at`, `source_types`, `status` (eligible, excluded), `exclusion_reason`	Inesh
`number_selections`	Which number a case will call, and why	`case_id`, `contact_point_id` or none, `candidates` (every number with its reason), `rule_version`	Inesh
`calls`	One call attempt	`case_id`, `contact_point_id`, `provider_call_id`, `status`, `recording_ref`, `transcript_ref` (redacted), `agent_version`	Sami
`call_answers`	One answer the agent recorded through a tool	`call_id`, `slot`, `value`, `tool_call_id`, `captured_at`	Sami
`decisions`	The decision engine's output	`case_id`, `outcome`, `rules_fired`, `inputs_hash`, `engine_version`	Abhi
`evidence_events`	The append-only log of everything above	`seq`, `chain_id`, `case_id`, `type`, `actor`, `body_jcs` (the exact canonical JSON that was hashed), `content_hash`, `prev_hash`, `hash`	Abhi

Plumbing tables (owned by Abhi): `case_transitions` (the allowed moves), `outbox` (commands waiting to run), `timers` (wake-ups for retries, callbacks and deadlines) and `inbound_webhooks` (raw signed deliveries, unique on event ID). Every table also carries a `dataset_id`, so a demo reset or an attack-lab scenario starts a fresh dataset instead of deleting rows.

Attack lab tables (owned by Sami)

Table	What a row is	Key fields
`config_versions`	A frozen version of every prompt, script, rule set and model choice	`kind`, `content_hash`, `git_sha`
`scenarios`	One attack or legitimate-but-odd change	`class`, `is_attack`, `held_out`, `author_model`, `expected_outcome`, `spec`
`lab_runs`	One full run of the suite	`config_version_ids`, `git_sha`, `started_at`, `cost_usd`, `duration_s`
`lab_results`	One scenario's result in one run	`run_id`, `scenario_id`, `observed_outcome`, `passed`, `failure_class`, `transcript_ref`

How the evidence log stays tamper-evident

- **Append only.** The app's database role can insert into `evidence_events` but has no update or delete rights; a trigger blocks both anyway.
- **Hash chain.** Each row stores `hash = SHA-256(prev_hash + SHA-256(the event's canonical JSON))`, so editing any old row breaks every hash after it.
- **Checked, not trusted.** A verify job recomputes the chain and the dashboard shows a green "chain intact" badge, or the first broken row.
- **Everything that matters is an event:** detection, hold, number selection with every candidate, call start and end, each recorded answer, the decision with the rules that fired, the release, and the versions of every prompt and rule in force.

Component design

Dialback has seven components, each owned by one person. For each one, this section gives its job, the interface the rest of the system calls, how it works inside, and how it fails. Shared request and response types live in `packages/contracts` (see Team split).

1. Ramp adapter and change detection (Abhi)

Job. Make every call to Ramp, and turn Ramp's signals into exactly one case per bank change.

Interface. One Python protocol, `RampPort`, with two implementations: `RampHttpAdapter` for Ramp's REST API and `RampMock` for our Ramp-shaped service. The `DIALBACK_RAMP` setting (`mock` or `sandbox`) picks one at startup, and no other code knows which one is running.

Method	Ramp endpoint	Notes
`get_vendor`	`GET /developer/v1/vendors/{vendor_id}`	The vendor record
`list_vendors_updated_since`	`GET /developer/v1/vendors?from_updated_at=…`	The poller's sweep
`list_vendor_accounts`	`GET /developer/v1/vendors/{vendor_id}/accounts`	Masked to the last four digits
`list_vendor_contacts`	`GET /developer/v1/vendors/{vendor_id}/contacts`	Contact edits also go to the Trust Ledger
`list_bills_for_vendor`	`GET /developer/v1/bills`	Pages of 100; filtered by vendor in code if the API can't do it
`place_vendor_hold`	`POST /developer/v1/vendors/{vendor_id}/hold`	Beta endpoint
`release_vendor_hold`	`POST /developer/v1/vendors/{vendor_id}/release`	Beta endpoint
`set_bill_payment_date`	Bill update (exact path to confirm)	Restores schedules after a release
`get_audit_events`	`GET /developer/v1/audit-logs/events`	Finds who made the change
`verify_webhook`	None (a local HMAC check)	Checks Ramp's signature

Detection: webhooks are hints, the poll diff is the truth.

1. A `vendors.updated` webhook arrives. The adapter checks `X-Ramp-Signature` (an HMAC-SHA256 of the raw body, compared in constant time) before parsing anything, stores the raw delivery in `inbound_webhooks` (unique on Ramp's event ID), and answers 2xx within Ramp's 10-second limit, for duplicates too.
2. The worker fetches the vendor's current bank accounts and compares them with `bank_accounts`, which is Dialback's own history, because Ramp has no history endpoint. A new account writes one `bank_change_events` row and opens a case.
3. A poller runs the same comparison for every vendor updated since its last sweep, plus a full sweep each night. This catches a change that never sent a webhook.
4. A change-request email forwarded to Dialback is the third way in. The Trust Ledger uses it to mark the change context.

5. Ramp's audit log tells us who made the change. When it can't, the changer is recorded as unknown and the release needs two approvers (see the decisions section).

Hold, in a fixed order. A Ramp vendor hold clears each bill's payment date, so Dialback saves the dates first.

1. Save every unpaid bill's `payment_date` to `holds.original_schedule`.
2. Place the vendor hold.
3. Read the bills back, confirm each one is on hold, and catch any bill created in between.
4. Emit `hold_confirmed`. If the read-back still fails after retries, emit `hold_failed`: the case goes to Needs review, and the dashboard shows a red "Hold not confirmed" banner.

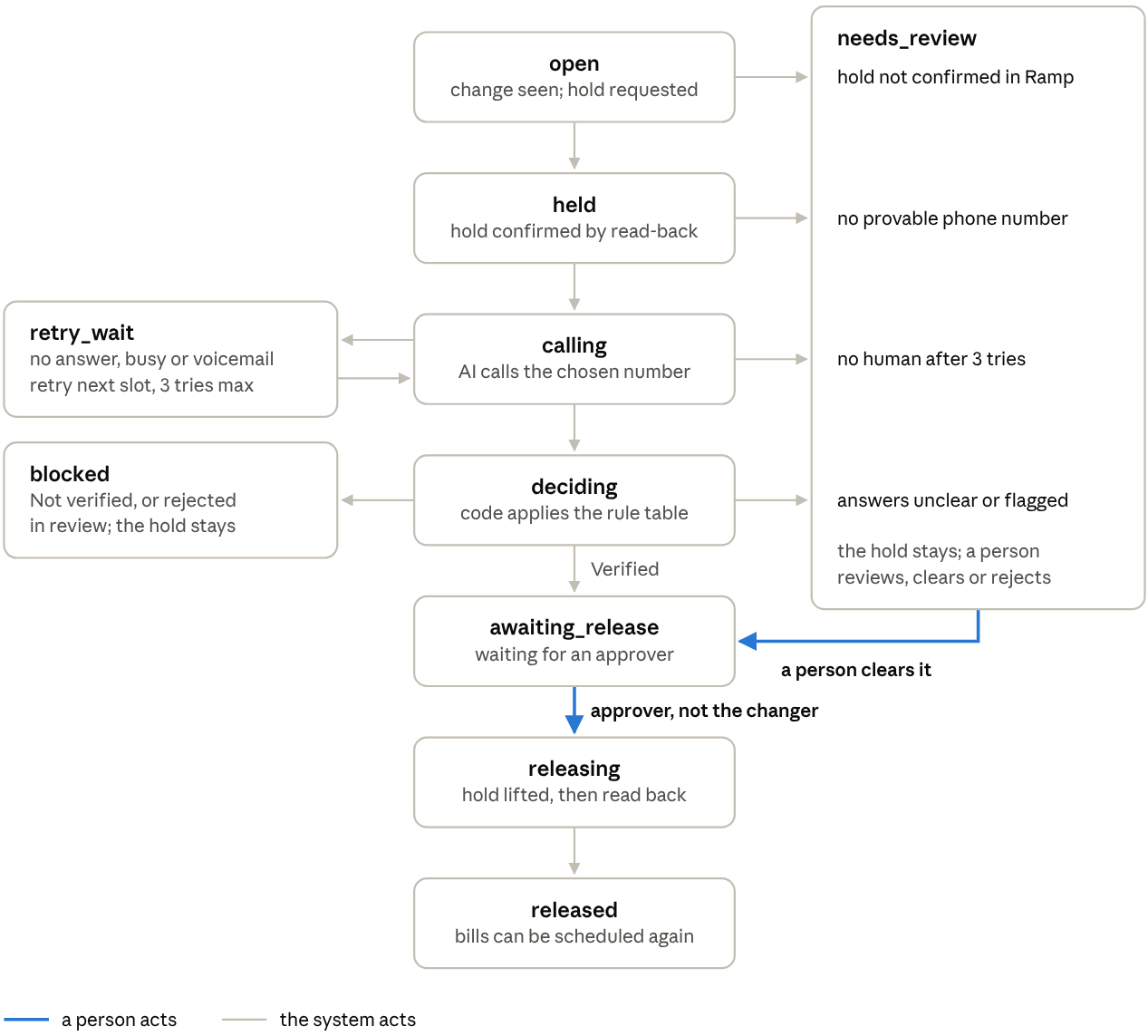
Release runs the same steps in reverse: release the hold, read it back, restore the saved payment dates, and let Ramp's own payment approvals run as usual.

Limits. A token bucket keeps Dialback under Ramp's limit of 200 requests per 10 seconds per IP, with backoff on a 429 response. The app asks only for read access to vendors, bills and audit logs, plus whatever the hold endpoints need, and never for a scope that can create or pay a bill.

The mock (`mock_ramp` schema) copies Ramp's paths, field names and behavior: holds that clear payment dates, releases that don't reschedule, thin signed webhooks with retries, a switch that duplicates and reorders them, a "silent change" mode that sends no webhook, last-four masking, and 429s past the rate limit.

2. Case orchestrator (Abhi)

Job. Move each case through its states, run each side effect exactly once, and make it impossible to skip a step.



A new bank change on the same vendor sends an unfinished case back to held and voids any earlier decision.

Case state machine · 10 states, 2 moves that need a person

Two moves need a person (the accent arrows), and every failure path ends in a state where the hold stays on.

One door for every change. Every state change goes through one SQL function, `apply_case_event(case_id, event_type, actor, idempotency_key, body)`. In a single transaction it:

1. looks up the move in `case_transitions` and checks that this kind of actor may make it (only the system confirms holds and records decisions; only a person clears a review or approves a release);
2. compares and sets `cases.state` and `cases.version`, so two writers can't both win;
3. appends the event to the evidence log through `append_evidence()`;
4. queues the follow-up commands in `outbox`: place the hold, pick a number, place a call, decide, release, notify.

If the same `idempotency_key` arrives twice, the second call returns "already applied", so every retry is safe.

Transition table. These are rows in `case_transitions`. CI redraws the diagram above from them, so the picture can't drift from the code.

From	Event	Who	Guard	To
(new)	`change_detected`	system	No unfinished case for this vendor	`open`
`open`	`hold_confirmed`	system	Read-back shows the hold	`held`
`open`	`hold_failed`	system	Retries used up	`needs_review`
`held`	`number_resolved`	system	The Trust Ledger rule passed	`calling`
`held`	`no_provable_number`	system	None	`needs_review`
`calling`	`call_ended`	system	Answers recorded	`deciding`
`calling`	`call_ended`	system	No person reached; tries left	`retry_wait`
`calling`	`call_ended` or `call_timeout`	system	No tries left	`needs_review`
`retry_wait`	`retry_timer_fired`	system	Inside the calling window	`calling`
`deciding`	`decision_made`	system	Verified	`awaiting_release`
`deciding`	`decision_made`	system	Not verified	`blocked`

From	Event	Who	Guard	To
`deciding`	`decision_made`	system	Needs review	`needs_review`
`needs_review`	`review_cleared`	person	Not the changer; written reason	`awaiting_release`
`needs_review`	`review_blocked`	person	Written reason	`blocked`
`awaiting_release`	`release_approved`	person	Approver role; not the changer, nor the reviewer who cleared it; case version matches	`releasing`
`releasing`	`hold_release_confirmed`	system	Read-back shows no hold	`released`
Any unfinished state after `held`	`change_detected`	system	Same vendor	`held`, version + 1, earlier decision void

Commands and timers. The core worker picks up due `outbox` rows with `FOR UPDATE SKIP LOCKED`, runs each one with an idempotency key (`{case_id}:hold`, `{case_id}:call:{n}`, `{case_id}:release`), and marks it done. A command that keeps failing stops after a capped number of attempts, and the case goes to Needs review. The `timers` table holds every wake-up: the next calling slot, a callback the vendor asked for, and the three-business-day deadline, after which the case goes to Needs review with the hold still on.

Evidence chain. The app turns each event into RFC 8785 canonical JSON and hashes it. `append_evidence()` re-checks that hash, then links the event to the previous one while holding a lock: `hash = SHA-256(prev_hash + content_hash)`. A unique constraint on `(chain_id, prev_hash)` rules out forks. A demo reset starts a new `chain_id` rather than deleting rows.

Background checks. Every five minutes, a reconciliation job confirms that every vendor with an unfinished case is still held in Ramp, and a verify job recomputes the hash chain. Either failure raises a top-priority alert on the dashboard.

Invariants. SQL enforces these, and property tests and the verify job check them.

- **I1.** A released case has an approval for its current version from a person who isn't the changer.
- **I2.** While a case is unfinished, its vendor is held in Ramp.

- **I3.** The only way into `released` is through `releasing`.
- **I4.** The AI emits no events. Code turns its tool calls into the answers inside `call_ended`.

Clock. Every component reads the time through one `Clock` interface. Production uses the real clock and the attack lab uses a simulated one, so rules like "90 days old" and "next business day" give the same result on every run.

3. Decision engine (Abhi)

Job. Turn a finished call into Verified, Not verified or Needs review, the same way every time.

Interface. `decide(facts: CaseFacts) -> Decision` is a pure function: no database, no network and no model. A `Decision` holds the outcome, the IDs of the rules that fired, `engine_version` and a hash of the inputs. It is stored in `decisions`.

Inputs come from three sources, and the AI fills only the second:

- **Telephony**, from Twilio: the number dialed, the call's direction, start and end times, and whether a person or a machine answered.
- **Recorded answers**, from the agent's `record_answer` tool: who answered, recording consent, the company they named, their name and role, whether they asked for the change, and the four digits they said.
- **Computed by code after the call:** whether the company matches the vendor's legal or trade names; whether the speaker first appeared in the change thread; whether one person gave both the "yes" and the digits; whether every recorded answer appears in the transcript turn it cites; and whether the disclosure came first.

Last-four check, tested in this order: `inconsistent` (two different answers), `ambiguous` (the old and new accounts end in the same four digits), `match`, `matches_old_account`, `near_miss` (one digit off, or two neighbors swapped) and `mismatch`. Fewer than four digits, "I don't know" or a refusal gives `not_provided`.

Rules. Blocks are evaluated in order, and the first one that matches decides.

1. **Not verified** (4 rules): the vendor says they asked for no change; the digits don't match; the digits match the old account; or the callee offers a number from the request. The hold stays and the controller is alerted.

2. **Needs review** (16 rules), for example: no live person after three tries, recording consent declined, the wrong company, a speaker who first appeared in the change thread, near-miss or missing digits, pressure such as "just approve it", an opt-out, or an answer the transcript doesn't support.
3. **Verified** (12 conditions, all required): from "the number was eligible and we dialed out", through "the right person confirmed the change and gave matching digits", to "no flags and no script violations".
4. **Anything else** is Needs review, including any error.

The rules are a versioned table in code, and each one has a test built from the call protocol's worked examples. A strict mode, which also requires a contact known from before the request, is built as well, so the scorecard can show what it costs in extra reviews.

People use the same engine. When a reviewer phones the vendor themselves, they enter the same answers in the dashboard, and the same `decide()` runs on them. Clearing a case that the engine sent to review takes two people, neither of them the changer: the reviewer and the approver.

4. Trust Ledger (Inesh)

Job. Record where every phone number in a vendor's history came from, then pick the number to call, or refuse to pick one.

Interface.

- `ingest(document, custody)` adds a document and the values found in it.
- `select_number(case)` returns the chosen number with up to two alternates, or `no_provable_number`, plus every candidate and why it was or wasn't eligible.
- `lineage(vendor_id)` feeds the dashboard's lineage panel.
- `risk_list()` feeds the opening screen of the demo.

Pipeline. Code runs every stage except labeling.

Stage	What happens
Custody	Record how the document reached us (email, contract upload, vendor portal, listing snapshot or paper mail), our own receive time, the sender, the thread headers and a SHA-256. Attachments inherit their email's custody.
Text	pdfplumber extracts the text along with each character's color, size and position. Pages without a text layer go to Claude's vision input.
Visibility	Flag white, tiny and off-page text. Hidden values are kept, marked as not visible to a person, and never eligible. OCR of the rendered page double-checks flagged documents, everything in a change context, and a random 10%.
Candidates	libphonenumber finds phone numbers. Code finds checksum-valid routing numbers and labeled account numbers.
Labeling (AI)	A quarantined model with no tools sees one document's visible text and returns JSON that must match a schema: each value's role (main, AR, billing, fax), its party (vendor, customer, bank) and the exact quote.
Grounding	Each quote must appear in the visible text and match a candidate. Code parses the value from the quote, not from the model's output; anything else is dropped.
Entities	Documents attach to vendors by tax ID and domain. Fuzzy name matches wait for a person to confirm them, and lookalike domains are never merged.
Observations	Append-only rows. `source_type` comes from custody metadata, never from the model.

Independent sources. A source type is how a value reached us, not what kind of document it was.

- All email counts as one source, however many mailboxes or domains it came from.
- A signed contract our staff stored, a vendor-portal entry, a public listing and paper mail each count once.
- The old vendor master's "number on file" is shown but never counted, because nobody knows where it came from.

The selection rule (`ledger-select-v1`). Pick the oldest number that:

- appears in at least two independent source types before the cutoff;
- was first seen at least 90 days before the cutoff;
- did not first appear in the change context;
- isn't one digit off, or two swapped digits away from, an older vendor number, unless both are long-established lines.

Ties go to the number with more source types, then to an AR or billing line over the main line. If nothing qualifies, the case goes to Needs review and no call is made. Alternates are used only if the first number is dead.

- **Cutoff:** the earliest of the request's arrival, the first message from a lookalike domain, and the change appearing in Ramp. All times are our own receive times, never a printed date or an email's `Date:` header, which a fraudster controls.
- **Change context**, deliberately broad: the request's thread, the same subject line within 30 days, anything from a lookalike domain, and anything received after the cutoff.
- **Replayable:** a selection reads only rows recorded before the case opened, and stores the rule version, its settings and a hash of the rows it used.

R2 in practice. A number is rejected if it first appeared in the change context. A long-established number that the thread happens to repeat stays eligible; otherwise every honest vendor's email signature would disqualify its own real number.

Synthetic company. A generator builds 500 vendors, 40 to 60 of them with years of invoices, contracts, emails and listing snapshots. It uses only fictional values: phone numbers in the reserved 555-0100 to 555-0199 range and domains under `.example`. Code plants every value, so the labels are exact, and keeps each vendor's real numbers several digits apart so the near-miss check doesn't fire on noise. Twelve poisoning patterns and five legitimate-but-odd controls have their expected outcomes frozen in git before any extractor code exists. The patterns include a one-digit-off number in a hijacked thread, a lookalike domain, slow poisoning over weeks, hidden PDF text, a backdated header and a poisoned "number on file".

Known limit. A fraudster who has controlled a vendor's mailbox and a public listing for more than 90 days, at a vendor with no stored contract, can get their number picked. The scorecard says so.

5. Voice agent (Sami)

Job. Make a short, disclosed call to the chosen number and record what the vendor says, with no way to change the outcome.

Interface. The core worker sends `{case_id, attempt}`. The voice service reads the chosen number from the database itself, so nothing in the request can change who gets called. When the call ends, it sends `call_ended` with the recorded answers back to the core. Calls that never connect (no answer, busy or failed) come back through Twilio's status callback, which is verified the same way.

One policy, two transports. `policy.step(state, event) -> Step` is the only dialogue logic. The Twilio ConversationRelay adapter feeds it the caller's words as text over a WebSocket and speaks its replies. The attack lab's text phone line calls the same function with no Twilio at all.

How a call runs.

1. **Before dialing, code checks** that the case is unfinished and held, the number is the Trust Ledger's pick, the number hasn't opted out, the vendor is inside its calling window (Monday to Friday, 9:00 to 16:30 vendor local time), and the caps aren't reached (3 attempts per case, 3 AI calls per number per 30 days). For the demo, the number must also be on the team's allowlist.
2. **Code mints a one-time `call_token`** tied to the case, the attempt and the number, and passes it to Twilio as a `<Parameter>`. When Twilio connects, the adapter checks `X-Twilio-Signature`, the token, and that the dialed number is the chosen one. The model never sees a case ID.
3. **The disclosure is fixed text from code.** It plays as the `welcomeGreeting` with interruptions turned off, so it always plays in full: an AI assistant calling for the customer's AP team, on a recorded line, with a callback number.
4. **A small state machine owns the order of questions:** the right company, the right department, the person's name and role, whether they asked to change how they're paid, the last four digits of the account they want paid, and a close that reveals nothing. The model only words the questions.

5. **A speech guard checks every reply before it's spoken.** It blocks runs of three or more digits (except a read-back of digits the callee just said), the words "verified", "approved" and "match", amounts and bank names, and speaks a fixed fallback line instead.

Tools. The model gets exactly three tools, plus a fourth inside phone menus.

Tool	What it does	Limits
`record_answer(slot, value, turn_id)`	Saves what the callee said	Slot names come from a fixed list and each value is checked against its slot's type; must cite the transcript turn; returns only `{"ok": true}`
`schedule_callback(window)`	Books a callback	Same number only, so there's no number field; code enforces the window and caps
`end_call(reason)`	Ends the call	Reason from a fixed list
`press_keys(digits)`	Picks a phone-menu option	Menu state only; 1 to 3 keys from 0–9, `*` and `#`; 5 uses

Arguments are validated strictly, with no extra fields allowed. An unknown tool or a bad argument is logged as a policy violation and the question is asked again; a second one ends the call and sends the case to Needs review.

What the model never sees: the new or old bank details, the expected digits, the request's content or date, or the number's lineage. Its context holds the customer's name, the vendor's name, the AP callback number and the script. The callee's words are passed in as quoted data, never as instructions.

Edge cases. Voicemail gets one fixed, generic message per case (no bank, change or amount), then a retry. A phone menu ends the call and sends the case to Needs review for Nov 10; pressing through a mock menu is a stretch goal. A dropped call gets one redial within five minutes, and its answers are never combined with the redial's. "Call my cell instead" is recorded and never dialed.

Setup and fallback. Twilio's free trial blocks ConversationRelay, so the account is upgraded on day 1. If live calls aren't reliable by the Nov 4 go/no-go, the same script and tools move to Retell, with the case ID kept out of the prompt in Retell's call metadata.

6. Attack lab (Sami)

Job. Replay hundreds of scams and legitimate-but-odd changes through the real pipeline, and report how often Dialback is fooled or wrongly blocks a real vendor.

Interface. A command-line tool. `lab run --split dev --epochs 1 --max-usd N` writes `results.jsonl` and `scorecard.json`, and exits with an error when a hard gate fails, which is what CI checks.

Scenarios. One YAML file per scenario family (`dialback.scenario/v1`), with variants. A scenario states facts, not answers: who really owns each phone number, what the fraudster controls (mailbox, documents, phone of record), whether the change is real, and which documents the ledger can find. An oracle derives what must happen from those facts: never dial a line the fraudster controls, never end an attack Verified, and end a reachable legitimate change Verified. Expected outcomes are frozen with a hash before a run.

How a run works.

1. Each scenario gets its own dataset ID and simulated clock, so scenarios run in parallel and time-based rules reproduce.
2. The change comes in through a real door: a mock webhook, a silent change the poller must find, or a forwarded email.
3. The real pipeline runs: detection, hold, Trust Ledger, call and decision.
4. The text phone line answers based on the number dialed: a fraudster persona, a real-vendor persona, or a scripted line (no answer, voicemail or phone menu). Personas run on a different model family from the agent.
5. A release bot, a lab user who isn't the changer, approves everything the console allows. That makes "released to fraud" something we measure, not assume.
6. Outcomes are read through the same API the dashboard uses, and the evidence chain is re-verified.

Scoring. Code checks the hard bars: the outcome, the number dialed, disclosure first, no account digits spoken, and an intact chain. An LLM judge from a third model family grades only the parts of script adherence that code can't check, and a person spot-checks 10% of its labels.

Scorecard. False Verified (bar: 0), released to fraud (bar: 0), missed detections by attack class, false holds and the human-review rate on legitimate changes, script adherence, text-versus-audio agreement, time to decision, and cost. Rates carry Wilson 95%

intervals, and a zero count carries its exact upper bound (0 out of 300 supports "below 1%"). Every number sits next to the count of seed families, because variants of one seed aren't independent.

CI gates. Any false Verified, release to fraud, call to a fraudster's line, script violation caught by code, or broken chain blocks the merge. A worse rate only comments on the pull request, because at this sample size a move from 5% to 10% can't be told apart from noise.

Audio spot checks. 10 to 20 of the same scenarios run over real phone audio, with teammates reading persona cards, and are reported as agreement with their text runs. The 300 text runs are never called "calls".

Held-out set. A teammate who didn't write the code under test writes 10 to 20 extra scenarios from class descriptions alone, using a different model family. Only their hash is committed until the final run.

7. Dashboard (all three)

Job. Show each case's evidence, take the release decision, and show the scorecard.

How it works. Next.js on Vercel, with Supabase Auth (email magic links) and roles for clerks, approvers, auditors and admins, plus a lab role for the release bot. Pages call only the core API, sending the signed-in user's token, and subscribe to Supabase Realtime for live updates. A release reaches `release_case()` under the approver's own token, so the database itself knows who is releasing. An admin manages users but can't release.

Page	What it shows	Owner
Case queue	Unfinished cases by state and age, and held bills near their due date	Abhi
Case detail	A timeline built from evidence events, the chain-intact badge, and the decision with the rules that fired	Abhi
Release console	The evidence summary, a required reason, and release or reject; the changer never sees the button, and SQL refuses them anyway	Abhi
Number lineage	The chosen number with its sources and first-seen dates, and every rejected number with its reason	Inesh

Page	What it shows	Owner
Vendor risk list	The opening screen: vendors whose number on file is provable, unproven, a near miss, or missing	Inesh
Call view	The transcript with recorded answers highlighted, the recording, and any speech-guard trips	Sami
Scorecard	The latest run's gates, rates with intervals, results by attack class, and what changed since the last run	Sami

Every sentence that explains a decision comes from a template in code, never from a model, and says "provable" or "not provable", never "safe".

8. Shared plumbing

- **Versions.** Every prompt, script, rule table, ledger setting and model choice is frozen in `config_versions` with a content hash, and every call, selection and decision records the versions it used.
- **Logs and traces.** Structured logs keep an allowlist of fields (IDs, states, counts, timings) and mask any run of six or more digits. LLM calls are traced in Langfuse, errors go to Sentry, and every line carries the `case_id`.
- **Alerts.** A hold not confirmed, an unfinished case whose vendor isn't held, a broken chain, a command stuck retrying, and any webhook with a bad signature.

Tech stack and key decisions

The stack

Layer	Choice	Notes
Dashboard	Next.js and TypeScript on Vercel	A preview deploy for every pull request
Backend	Python 3.13 with FastAPI and Pydantic, on Railway	Three services: core, voice and the Ramp mock
Data	Supabase: Postgres, Auth, Realtime, private Storage and Vault	Schemas by owner: `app`, `audit` and `mock_ramp` (Abhi), `ledger` (Inesh), `voice` and `lab` (Sami)

Layer	Choice	Notes
Background work	`outbox` and `timers` tables, run by the core worker	The worker also runs the poller, the reconciliation job and the chain check on a schedule
Telephony	Twilio Programmable Voice with ConversationRelay, on a paid account	Twilio turns speech into text and text into speech; our code only sees text
Documents	pdfplumber, pypdfium2, tesseract, python-phonenumbers, RapidFuzz	Local and free
Models	Claude for the document reader and the call agent, with pinned versions. The reader starts on Haiku 5.5, and a 100-document labeled set decides on day 2 whether it needs Sonnet 5.5	Attackers and personas run on a second model family and the judge on a third, so no model attacks or grades itself
Contracts	OpenAPI 3.1 and JSON Schema in `packages/contracts`, with generated TypeScript and Pydantic types and a Prism mock server	Frozen on day 1
Repo and CI	One monorepo (pnpm and Turborepo for TypeScript, uv for Python) on GitHub Actions	Lint, type, unit, contract and attack-lab smoke checks on every pull request
Observability	Langfuse for model calls, Sentry for errors, structured logs	Every record carries the `case_id`

Key decisions

Each decision is written up in the repo as an architecture decision record in Michael Nygard's format (`docs/adr/NNNN-title.md`, with title, status, context, decision and consequences). Records are numbered once and never reused, and a reversed decision is marked superseded rather than deleted. The short versions follow.

ADR-1. Postgres is the referee. The state machine, four-eyes release and evidence chain live in three SQL functions behind row-level security: `apply_case_event`, `release_case` and `append_evidence`. *Why:* these rules protect money, and in the database they hold even if a service has a bug, a server key leaks or a coding agent gets something wrong.

Trade-off: SQL functions are harder to write and test, so they stay this small. *Rejected:* rules only in application code; XState, which would run in the app where the database can't enforce it.

ADR-2. Python services on Railway, the dashboard on Vercel. *Why:* a live call holds a WebSocket open for minutes, and Vercel's WebSockets are in beta and close at 300 seconds on the Hobby plan. Python also has the document-parsing and eval libraries, and one backend language lets the attack lab import the real code. *Trade-off:* two platforms and two languages, held together by generated contracts. *Rejected:* everything on Vercel with Python as serverless functions, which leaves the call nowhere to live. Fly.io or Render would work as well as Railway.

ADR-3. Postgres tables instead of a workflow engine. Every wait (a calling window, a retry, a callback, a deadline, a person's release) is a case state plus a `timers` or `outbox` row that the core worker claims with `FOR UPDATE SKIP LOCKED`. *Why:* a case has about ten steps and the transition table already guards each one, so an engine would add a second source of state without adding safety, and there's no new service to learn or host. *Trade-off:* we write retries, backoff and timeouts ourselves, and there's no built-in run viewer; the evidence log is the history. *Rejected:* Vercel Workflows, because the voice service needs a long-running host anyway; Temporal, which Ramp runs internally and which is the production path, but whose workers and determinism rules are too much for ten days; Inngest, whose free tier ends a run after 30 days, waiting time included.

ADR-4. AI collects, code decides. Models only label documents and hold conversations. A pure function decides the outcome, and only code can pick a number, place a hold or lift one. *Why:* outcomes are predictable, testable and explainable, and no one can talk the system into a verdict, because no model has a tool that sets one. This follows the published design patterns for prompt-injection safety (action selector, dual LLM and context minimization). *Trade-off:* the script is rigid, so unusual but honest calls land in Needs review, and the scorecard counts them. *Rejected:* an LLM grading the call.

ADR-5. Our own dialogue loop on Twilio ConversationRelay. *Why:* the same `policy.step()` runs on live calls and in the lab, so 300 text runs test the real agent. Text in and text out is what makes the speech guard possible, and because our code owns the tool path, the model never sees a case ID. *Trade-off:* we build the WebSocket server, reconnects and turn-taking tuning ourselves. *Fallback:* Retell, if live calls aren't reliable by the Nov 4 go/no-go. *Rejected:* Vapi and ElevenLabs Agents, whose hosted loops would let the text lab drift from the live agent; OpenAI's speech-to-speech models, which leave no text step for the speech guard and need organization-level enablement for outbound calls.

ADR-6. Ramp's REST holds, with polling as the truth for changes. *Why:* Ramp has no bank-account history endpoint, and it isn't confirmed that `vendors.updated` fires on bank changes, so Dialback compares accounts itself and treats a webhook as a nudge to check sooner. A Ramp hold stops payments inside Ramp, so Dialback never has to touch money. *Trade-off:* the hold endpoints are in beta, a hold clears payment dates (Dialback saves them first), and polling spends rate limit. *Rejected:* bill-level holds alone, which miss new bills; Ramp's agent tools (`ramp_hold`) through an agent identity, which is in early access.

ADR-7. A hash-chained evidence log. Each event is RFC 8785 canonical JSON, hashed with SHA-256 and linked to the one before it in SQL. *Why:* auditors and insurers need evidence they can check. One crime-insurance endorsement we found (Intact, Form M350) covers a loss only if the callback was "recorded, logged or otherwise documented". *Trade-off:* the chain is tamper-evident, not tamper-proof. Someone with owner rights could rewrite the whole chain consistently, so the latest hash is committed to GitHub with each scorecard as an outside witness. *Production path:* a daily Merkle root, timestamped by an outside authority.

ADR-8. Last four digits plus a keyed fingerprint. For each bank account, Dialback stores the routing number, the last four digits and `HMAC-SHA256(key, routing + account)` , never the full number. *Why:* that's enough to compare accounts and to ask "have we seen this account before?", and a leak exposes nothing usable. A plain hash wouldn't do, because account numbers are short enough to brute-force. *Trade-off:* rotating the key means fingerprinting again, so each fingerprint records its key version.

ADR-9. Four-eyes in SQL, with the user's own token. `release_case()` takes the caller from `auth.uid()` and checks the approver role, that the caller isn't the changer, and that the case version matches what they saw. *Why:* segregation of duties has to hold even if the dashboard or API has a bug, so the server key, which bypasses row-level security, never touches a human decision. *Trade-off:* the core API passes the user's token through to the database.

ADR-10. An unknown changer means two approvers. A change the vendor supplied, or one synced from an ERP, may have no Dialback user behind it, so "not the changer" can't be checked. In that case, release needs two different approvers. *Rejected:* leaving such cases in Needs review, which is safe but turns every vendor-supplied change into manual work.

ADR-11. The attack lab runs production code on a simulated clock. *Why:* a lab that tests a copy proves nothing. The lab imports the real packages, sends changes through the real doors, reads outcomes through the real API and scores from case state. *Trade-off:* every component must read time from `Clock` and respect `dataset_id`, and a 20-scenario smoke run goes through the deployed system to catch drift. *Rejected:* hosted voice simulators and red-team frameworks (promptfoo, PyRIT) as the harness, because none of them can see our case state. We borrow their attack ideas instead.

ADR-12. Contracts first. Every interface between owners is OpenAPI 3.1 and JSON Schema in `packages/contracts`, frozen on day 1, with generated types and a Prism mock server. *Why:* three people and their coding agents can build in parallel against mocks from the first hour, and CI catches drift between the spec and the code. *Trade-off:* changing a contract takes its own pull request, approved by every affected owner.

ADR-13. Sources are counted by how they reached us, and all email counts once. *Why:* a fraudster who takes over a vendor's email controls every mailbox on that domain and can register lookalike domains at will, so email can't vouch for itself twice. *Trade-off:* an honest vendor known only by email can't be called and goes to Needs review. The scorecard reports the looser variants (each mailbox counts once, or each domain counts once) side by side.

ADR-14. PDFs are read locally, with per-character checks. The text layer comes from pdfplumber, not an AI OCR service. *Why:* in our test, hidden white and 1-point text came through every text extractor we tried, and only pdfplumber's per-character color, size and position data flagged all three tricks (white, tiny and off-page). A model given raw text can't tell what a person would see. *Trade-off:* scanned pages need a separate path through Claude's vision input, so scans stay at about 5% of the corpus.

Security and safety

Dialback's adversary controls some channels (a vendor's mailbox, the documents it sends, sometimes a public listing, and whoever answers a phone) and wants one payment to reach their account. The design assumes every input from outside is hostile, and keeps every consequential decision where that input can't reach: in code, in the database, or with a person.

Threat model (STRIDE)

Threat	In Dialback	Control
Spoofing	A spoofed change email, an impostor on the call, a forged webhook, a spoofed incoming call	Never call a number first seen in the request (R2); the vendor must state the last four digits; HMAC-signed webhooks; outbound calls only, since an incoming call never counts
Tampering	Edited evidence, an altered document, a quietly changed rule	The hash chain with an outside witness; each document's SHA-256 in the log; the rule version stored with every decision
Repudiation	"I never released that." "The AI decided."	Approvals carry the user's ID and the case version, inside the chain; decisions store the code version and a hash of their inputs
Information disclosure	Bank numbers, transcripts, recordings, team phone numbers	Last four digits plus a keyed fingerprint; private storage behind row-level security; redacted transcripts; secrets kept out of the repo
Denial of service	A flood of fake change requests puts every vendor on hold; runaway call or model costs	Rate limits on intake; one case per vendor; one-click, logged duplicate marking; dollar caps
Elevation of privilege	A clerk releases their own change; a server key is used for a release; injected text reaches a tool	Four-eyes in SQL; release only through `release_case()` with the user's token; the agent has no tool that changes anything that matters

AI risks (OWASP Top 10 for LLM Applications, 2025)

Risk	Where it could hit	Control
LLM01 Prompt injection	PDFs, emails and signatures; the callee's words	The reader is quarantined with no tools; the voice tools only record; code decides (R9)

Risk	Where it could hit	Control
LLM02 Sensitive information disclosure	The agent is talked into revealing what it checks	The agent never holds the new details or the expected digits (R4); traces are redacted
LLM03 Supply chain	Libraries and model versions change under us	Lockfiles; pinned model versions; no third-party MCP servers in the case path
LLM04 Data and model poisoning	Phone numbers planted in a vendor's history	Two independent sources that predate the request, checked in code; near-miss and change-context exclusions
LLM05 Improper output handling	Extracted text shown in the dashboard or used in queries	Schema validation; all document text escaped in the UI; parameterized SQL only
LLM06 Excessive agency	An agent that could mark a case verified or release it	No such tool exists; holds and releases are code-only and gated by people
LLM07 System prompt leakage	A callee asks for the agent's instructions	Assume it leaks: the prompt holds no secrets, expected values or thresholds
LLM09 Misinformation	The reader invents or misreads a number	Grounding: a value must appear in the visible text, or it's dropped
LLM10 Unbounded consumption	Lab runs, long calls, retry storms	Dollar caps per run; call-length and attempt caps; rate limits on intake

LLM08 (vector and embedding weaknesses) doesn't apply, because no decision uses embeddings.

Prompt injection, by design. Dialback doesn't depend on any model resisting injection. It applies the design patterns from Beurer-Kellner et al. (2025):

- **Action selector:** the call agent's whole action space is three tools that only record.
- **Plan, then execute:** the script and tools are fixed before the call, so nothing the callee says can add a step.

- **Map-reduce:** the reader sees one document at a time, with no context from the others, and code combines the results.
- **Dual LLM, made stricter:** the reader is quarantined, and the privileged side is plain code rather than another model.
- **Context minimization:** the agent never sees the change thread, the new account or the number's lineage.

Even if a poisoned document convinces the reader that "the verified number is X", X is tagged as first seen in the change context, and code excludes it. DeepMind's CaMeL paper shows that a quarantined model can still be steered into returning an attacker's value, which is why provenance comes from custody metadata and never from a model.

Data protection

- **Bank details:** the routing number, last four digits and keyed fingerprint only (ADR-8). The key lives in Supabase Vault or a server-only setting.
- **Spoken digits** are compared in memory. The record keeps the result (`match`, `near_miss` and so on) and a salted hash, never the digits.
- **Recordings and raw transcripts** live in a private bucket that approvers and auditors open through short-lived signed links. Each recording's SHA-256 goes into the chain, and the dashboard and traces get a redacted transcript.
- **Logs** keep an allowlist of fields, mask any run of six or more digits and never include request bodies.
- **Secrets** stay server-side, with separate keys for the mock, the sandbox and the demo, and the narrowest Ramp scopes. Secret scanning runs on the repo, and every key is rotated after Nov 19.
- **Team phone numbers and consent records** never enter the repo.

Call compliance for the demo

1. A dial allowlist in code: only Abhi's, Inesh's and Sami's phones and team-owned test numbers, with a test for the guard itself.
2. Written consent from each teammate, and from anyone else who answers (such as a judge), covering AI calls, recording, processing by our vendors and use in the video. The records stay out of the repo.
3. The disclosure comes in the first sentence (AI, recording, the customer's name and a callback number), followed by a consent question. A "no" ends the call.
4. Every call is treated as needing all-party consent, because California, where the final is held, requires it for recording.

5. Nothing reaches anyone outside the team: no voicemails, phone-menu tests or lab audio.
6. Nothing is ever marketed on a call, which would turn it into telemarketing.
7. At the live final, a judge playing the vendor uses a team phone, after giving a spoken OK.

What production would also need (not built for the hackathon). The FCC ruled in February 2024 that AI voices count as "artificial" under the TCPA, so AI calls to cell phones need prior express consent. A real deployment needs counsel's sign-off on the TCPA, state recording laws, California's Invasion of Privacy Act and Public Utilities Code §2874, and state AI-disclosure laws. It also needs the customer contracted as the calling party, a consent line in vendor onboarding, an opt-out suppression list, voice and model vendors contracted not to train on call data, and human callers for numbers outside the US.

Testing and evals

Three kinds of proof. Tests show the code does what this document says. Evals measure how well the AI parts do their jobs. The attack lab (Component design, section 6) measures the whole system against fraud. The bars that protect money block a merge; everything else is reported.

Test layers

Layer	What it proves	Examples	When
Unit	Each rule matches the tables in this doc	One test per decision rule and per worked example; the last-four comparison; each clause of the selection rule; the speech guard; tool-argument validation	Every PR
Property	The invariants hold for any sequence of inputs	Random event sequences never break I1 to I4; the six Trust Ledger properties below	Every PR

Layer	What it proves	Examples	When
Database	The SQL refuses what it must	`release_case()` refuses the changer, a non-approver and a stale version; the evidence table refuses updates and deletes; a row edited by hand makes the verify job point at it	Every PR
Contract	Services match the shared spec	The specs validate; generated types don't drift; every example validates in both languages; nightly, the real API runs behind `prism proxy`	Every PR and nightly
Hash vectors	Python and SQL build the same chain	Shared golden vectors for canonical JSON and chain hashes	Every PR
End-to-end smoke	The whole path still runs	Seed the demo data and run one case from webhook to release in text mode	Every PR from Nov 2
Evals	The AI parts are good enough	Reader extraction and grounding, number selection, script adherence	Nightly
Attack lab	The system resists fraud	40 to 60 scenarios per PR; the full set nightly, repeated	Every PR and nightly

Trust Ledger properties, tested with Hypothesis:

1. Adding observations inside the change context never makes a number eligible.
2. The outcome depends only on rows recorded before the case opened.
3. Shuffling the observations doesn't change the outcome.
4. No number whose earliest dated sighting is in the change context is ever selected.
5. Role and party labels can reorder or remove eligible numbers, but never make a number with fewer than two source types eligible.
6. No selected number is one digit off an older candidate unless it is an established sibling line.

Ledger evals (Inesh) run against labels written by the same code that planted each value, plus 100 documents checked by hand.

What	Measure	Bar
Extraction	Precision, recall and F1 by kind of document (born-digital, scanned, signature, hidden text); role accuracy	Reported
Grounding	Share of model outputs rejected; ungrounded values accepted	0 accepted, by construction
Visibility	Share of planted hidden values flagged; agreement between the character checks and OCR	Reported
Poison detection	Precision and recall for each poisoning pattern, on the frozen and held-out sets	Every miss listed
Selection	Poisoned numbers dialed; correct refusals; false refusals on the controls	0 poisoned dials
Ablations	Minimum age of 0, 30, 90 or 180 days; near-miss distance of 1 or 2; the three ways of counting email	A trade-off chart

Script adherence is checked in code first: the disclosure is in the first turn, no account digits are spoken, no new number is accepted, and an opt-out ends the call. An LLM judge from a third model family covers only what code can't, such as a paraphrased hint about what is being checked, and a person checks 10% of its labels.

What we can claim. With zero failures in n independent runs, the 95% upper bound on the true failure rate is:

Runs with zero failures	Upper bound (exact, one-sided)
20	13.9%
60	4.9%
100	3.0%
300	0.99%

So 0 false Verified out of 300 attacks supports "below 1%", but only if the runs are independent. If those 300 come from 60 seed families, the honest bound is 4.9%, and the scorecard prints both. Rates carry Wilson intervals, and at these sample sizes a shift in a rate gates nothing; only hard failures block.

CI jobs (GitHub Actions)

Job	Trigger	Runs	Blocks merge
`contracts`	Every PR	Validate specs, regenerate types, fail on drift, validate examples	Yes
`ts`	Every PR	Lint, typecheck, test, build	Yes
`py`	Every PR	Lint, typecheck, pytest with the property tests and reader-injection fixtures	Yes
`db`	Every PR	Apply migrations to a fresh Postgres, then run the SQL tests	Yes
`boundaries`	Every PR	No imports between apps, no edits to generated code, no real-looking personal data in fixtures	Yes
`lab-smoke`	PRs that touch the agent, ledger, decision engine, prompts or model pins	40 to 60 scenarios in text mode, with the hard gates	Yes
`demo-smoke`	Every PR from Nov 2	One scripted case end to end	Yes
`lab-full`	Nightly, and on release tags	The full dev and held-out sets, repeated, with the scorecard saved to Supabase	Blocks tags
`contract-proxy`	Nightly	The real API behind `prism proxy`	Warns

Every lab job runs with a dollar cap, and API keys live in GitHub Actions secrets.

Team split

Each person owns a vertical slice: the service code, database schema, dashboard pages, tests and eval data for one part of the product. Slices meet only at the contracts in `packages/contracts`, so three people and their coding agents rarely touch the same files.

	Abhi	Inesh	Sami
Slice	The money path	The evidence	The call and the adversary
Builds	Ramp adapter and mock; detection and poller; case orchestrator, worker and SQL rules; decision engine; evidence chain; auth, roles and release	Synthetic company and its documents; ingestion, extraction and grounding; entity resolution; observations and number selection; risk list; ledger evals	Dialogue policy, tools and speech guard; Twilio setup and the ConversationRelay adapter; text phone line and personas; lab runner, oracle and scorecard; audio spot checks
Database schemas	`app`, `audit`, `mock_ramp`	`ledger`	`voice`, `lab`
Dashboard pages	Shell and navigation, case queue, case detail, release console	Number lineage, vendor risk list	Call view, scorecard
Attacks a teammate's part	Poisoned-document scenarios against the Trust Ledger	Impostor-caller scenarios against the voice agent	Silent-change, duplicate-webhook and release-race scenarios against the money path

Abhi's slice is the widest. If it is behind at the Nov 4 checkpoint, the decision engine, a pure function with its own tests, moves to Inesh, who already owns the other rule engine.

Where the slices meet

Between	Interface	Contract
Dashboard and core	REST with the user's token	`openapi/dialback.yaml`, served by Prism until each endpoint lands
Core and Trust Ledger	`select_number(case)` and `lineage(vendor_id)`, as Python calls	`schemas/selection.schema.json`
Core and voice	`POST /calls`, then `call_ended` back to the core	`openapi/internal.yaml`
Voice and decision engine	The answer slots and their allowed values, shared by the agent's tools, the text line and `decide()`	`schemas/call-answers.schema.json`
Every component and the evidence log	The event envelope that `append_evidence()` hashes	`schemas/evidence-event.schema.json`
Attack lab and everything else	Production's doors and public API, plus the scenario, result and scorecard formats	The `scenario`, `run-result` and `scorecard` schemas

Repo layout

```
dialback/
  apps/
    web/          Next.js dashboard; pages by owner             all
    core/         FastAPI API and case worker                    Abhi
    voice/        WebSocket server and Twilio adapter            Sami
    ramp-mock/    Ramp-shaped mock and webhook sender            Abhi
  packages/
    contracts/    OpenAPI, JSON Schema, examples, generated types all
    decision/     decide() and the rule table                    Abhi
    evidence/     canonical JSON and the hash chain              Abhi
    ledger/       ingestion, extraction, number selection        Inesh
    policy/       policy.step(), tools, speech guard             Sami
  corpus/         synthetic company, poison cases, frozen truth  Inesh
  evals/          runner, oracle, scorers, personas, scenarios   Sami
  supabase/       migrations/<schema>/ and seed/                 each
schema's owner
```

```
specs/          one short spec per feature, for the agents
docs/           design.md, adr/, c4/, sequences/, threat-model.md
.github/        workflows/, CODEOWNERS, pull_request_template.md
.claude/        settings.json and agents/reviewer.md
AGENTS.md  CLAUDE.md  pnpm-workspace.yaml  turbo.json  pyproject.toml
```

The held-out scenarios are not in the repo. They live in a CI secret or a private bucket, and only nightly and tag runs load them.

Building with coding agents

- **`AGENTS.md`** at the root holds the rules R1 to R9, the exact commands, who owns what, a short "never" list and what done means. Claude Code reads `CLAUDE.md`, not `AGENTS.md`, so `CLAUDE.md` contains the line `@AGENTS.md` plus any Claude-only notes. Each app and package has its own short `AGENTS.md` with its scope.
- **One loop per task:** write a short spec with an acceptance list in `specs/`; have the agent write failing tests first; implement in a fresh session until they pass; review the diff against the spec in another fresh session, with a reviewer agent that has read-only tools; then open a small PR that lists the commands run and their output.
- **Plan mode** for the SQL functions, the selection rule and the decision engine.
- **Parallel sessions:** one to three per person, each in its own git worktree (`claude -w <branch>`), and never two on one branch.
- **Hard limits, enforced:** agents can't read `.env` files, a hook blocks edits to generated contract code, and a stop hook runs the area's typecheck.
- **Agents open PRs but never merge.** A person approves every merge.

PR rules

- `main` is protected: PRs only, squash merges, and the required checks from the Testing section.
- Each PR needs one approval from a code owner who isn't the author. Every path has two owners, so the author's partner can approve.
- A contract change goes in its own PR, approved by both other teammates. From Nov 2, contracts only grow (new optional fields and new endpoints), and a breaking change needs a five-minute call between all three.
- PRs stay under about 400 changed lines, not counting generated code and fixtures.
- The PR template asks for what and why, the spec link, the commands run, the scorecard change if lab-relevant code moved, and screenshots for UI.

- **CODEOWNERS caveat:** required code-owner reviews work on public repos with GitHub Free, but private repos need a paid plan. Decide on day 1: make the repo public (it gets submitted anyway), or treat CODEOWNERS as advisory.

Daily rhythm

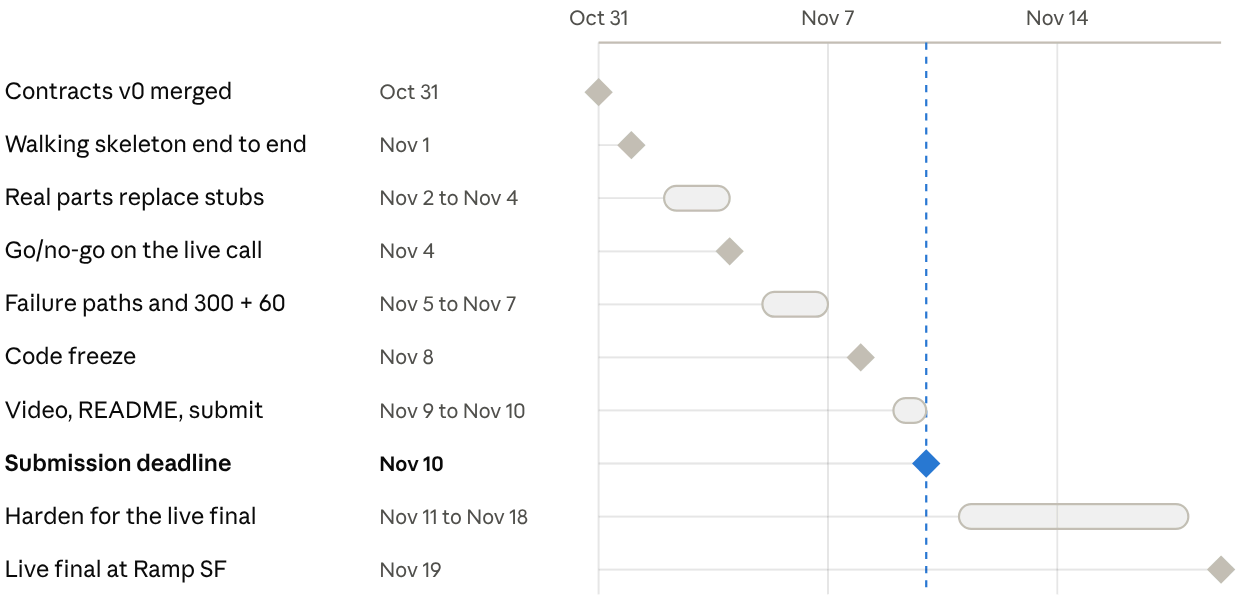
- Everyone merges small PRs to `main` at least once a day, and rebases daily.
- `main` must run the demo every evening.
- A 15-minute sync at a fixed time covers four things: is `main` green, how the scorecard moved, pending contract PRs, and blockers.
- Unfinished work merges behind flags, read in one module per language, and any unknown value falls back to the safe path: `DIALBACK_RAMP` (mock or sandbox), `DIALBACK_VOICE` (text or live), `DIALBACK_READER` (stub or llm), `DIALBACK_RESOLVER` (stub or ledger) and `DIALBACK_LAB_ATTACKER` (scripted or llm).

Build plan

Ten days to the Nov 10 submission (due 11:59 PM PT), then nine more to the live final at Ramp's San Francisco office on Nov 19. The plan tackles the riskiest pieces first (contracts, the Twilio account, Ramp access) and has a working path end to end from day 2, so every later day improves something that already runs.

Code freezes two days before the Nov 10 deadline

Build plan from Oct 31 to the live final on Nov 19



Build timeline · Oct 31 to Nov 19, 5 gates

The video is recorded the day after the code freeze, and the target is to submit by midday on Nov 10, leaving half a day of buffer.

Day by day

Day	Team milestone	Abhi	Inesh	Sami
Sat Oct 31	Contracts v0 merged	Repo, CI and `AGENTS.md`; Supabase, Railway and Vercel projects; transition-table and evidence-chain migrations	Generator v0; poisoning and control cases written, with their expected outcomes frozen and hashed	Twilio upgraded, two numbers bought, one test ConversationRelay call to a teammate; scenario format and oracle rules
Sun Nov 1	Walking skeleton end to end	Mock webhook → case → hold → stub decision → release console	A `select_number` stub that returns contract examples; demo seed v0	Text phone line with a scripted vendor; the runner runs 3 scenarios and writes a scorecard

Day	Team milestone	Abhi	Inesh	Sami
Mon Nov 2	Contracts additive-only; PR smoke gate on	Poller and diff; hold read-back and reconciliation	Reader with grounding and visibility checks; injection fixtures	40 dev scenarios; deterministic checks
Tue Nov 3	First live call between team phones	`release_case()` with four-eyes tests; chain verify job and badge	Selection rule with its property tests; entity resolution	Voice agent v1: disclosure, tools, last-four capture
Wed Nov 4	Go/no-go on the live call	Decision engine, with a test per rule	Lineage panel	Attacker personas on a second model family for 5 classes
Thu Nov 5	Failure paths; cost measured	Retry timers, deadlines and the unknown-changer path	Risk list for the opening screen; ledger evals	Voicemail, no answer and opt-out; 150 attacks and 30 legitimate changes; nightly full runs start
Fri Nov 6	Held-out set sealed	Chaos mode in the mock; hold-failure paths; held-out scenarios	Ablations; 100 documents checked by hand; held-out scenarios	Fixes from the first full runs
Sat Nov 7	Feature freeze	Queue and case-detail polish	Fixes from the ledger evals	300 attacks and 60 legitimate changes complete; 10 to 20 audio spot checks
Sun Nov 8	Code freeze at night	Demo mode and reset	Demo seed frozen	Final scorecard on the frozen commit; held-out results unsealed
Mon Nov 9	Video recorded	README, docs and ADRs	Lineage walkthrough for the video	Live-attack beat rehearsed

Day	Team milestone	Abhi	Inesh	Sami
Tue Nov 10	Submitted by midday PT	Submit the repo and video	Check that the README works from a clean clone	Check the scorecard and video links

Go/no-go on Nov 4. A live call between team phones must play the disclosure in full, capture the four digits correctly in at least four of five tries, end cleanly, and land its decision in the dashboard. If it doesn't, the live call moves to Retell with the same script and tools, and the lab stays on the text line. The same checkpoint rebalances work between the three slices.

The demo (recorded Nov 9, then live on Nov 19)

1. Open on the vendor risk list: how many vendors' "number on file" can't be traced to any document.
2. A bank-change email arrives for one vendor, and payments to it go on hold.
3. The lineage panel shows which number Dialback will call and why, and why the number in the email is rejected.
4. The live call: a teammate answers on a team phone, the AI discloses itself, and the vendor confirms.
5. Code decides Verified. The person who entered the change can't release it, so a second person does.
6. The evidence log: one row edited by hand, and the verify job points straight at it.
7. The scorecard: 300 attacks with the false-Verified count and its bound, plus one attack run live.

Recorded clips of each step are the backup if the phone network or Wi-Fi fails on stage.

Nov 11 to 18: getting ready for the live final

- Switch to Ramp's sandbox behind `DIALBACK_RAMP` if access arrives.
- Phone-menu navigation against a mock menu (stretch goal).
- More audio spot checks, and a second held-out set.
- Rehearse the live demo with a teammate playing a judge on a team phone.
- After Nov 19, rotate every key and delete raw recordings.

Risks, open questions and future plug-ins

Top risks

Risk	What we do about it	Owner
No Ramp sandbox, or the beta hold endpoints aren't enabled	The mock copies Ramp's paths, fields and behavior, so switching later is one setting; the video says plainly what ran against the mock	Abhi
The live call fails or is flagged as spam	Twilio upgraded and a test call on day 1; the Nov 4 go/no-go; Retell as the fallback; recorded clips as the backup	Sami
Digits are misheard	One repeat; near misses go to Needs review; speech hints for digits; text-versus-audio agreement measured	Sami
Honest vendors get held too often	False-hold and human-review rates on the scorecard; standard and strict modes compared	Sami and Abhi
A coding agent weakens a test or a rule	The rules live in SQL and property tests; `AGENTS.md` forbids weakening tests; CI gates; a person approves every merge	All
The worker and the database disagree	Compare-and-set transitions, idempotency keys and the reconciliation job	Abhi
Free tiers or model spend run out	A dollar cap on every lab run; cost measured on Nov 5; usage checked daily	Sami
A model provider flags attack generation	Fictional scenarios at modest volume; a separate API account from the one the coding agents use; a backup attacker model family	Sami
Phone numbers or keys leak from a public repo	Environment variables and Vault only; secret scanning; every key rotated after Nov 19	All

Risk	What we do about it	Owner
A demo reset fights the append-only log	Every reset starts a new dataset and chain, and nothing is deleted	Abhi
Abhi's slice is the widest	The decision engine moves to Inesh at the Nov 4 checkpoint if needed	All

Questions for the organizers

1. Can we get a Ramp sandbox business with an Admin user, or should we use the self-serve sandbox form?
2. Are the beta vendor hold and release endpoints enabled for sandbox apps, and which scope do they need?
3. Can we get an agent identity to use `ramp_hold`, or are the REST hold endpoints fine?
4. Does `vendors.updated` fire when a payee's bank account changes, whether the change comes from the dashboard, the API, the vendor or an ERP sync?
5. What is the audit-log event type for a bank-account change, and is `actor_id` the user who made the edit?
6. Can a sandbox bill be run through to "paid", so the demo can show a held bill not being paid?
7. Is it fine to show Ramp's sandbox and name in the video, and to run the attack lab against the sandbox, given its rate limit?

To look up ourselves, with Ramp's Developer MCP or a sandbox call: the exact schemas for vendor accounts, contacts and bills; the hold and release responses; the webhook verification path and signature encoding; whether listing bills can filter by vendor; and the path for updating a bill's payment date.

Future plug-ins. Each one attaches to a part that already exists, so none of them changes the core design.

Feature	What it adds	Where it plugs in
Recovery packet	If money already went out: a bank recall request and an IC3 complaint, prefilled from the evidence log	A new command when a case is `blocked`

Feature	What it adds	Where it plugs in
Insurer evidence export	The evidence log mapped to crime-policy wording, such as "pre-arranged callback"	A reader over the evidence chain
Nacha evidence report	A record that each bank change went through a risk-based check, for the fraud-monitoring rule that has covered every business ACH originator since Jun 19, 2026; it shows a process, never claims compliance	A report over cases and decisions
Staff drills	Fake change requests sent to AP staff, to see who follows the process	The attack lab's scenarios plus email intake
Network alarm	A warning to other Ramp customers who pay the same vendor when its mailbox looks compromised	Not-verified decisions, shared through Ramp
Account-name match	A bank's check that the account holder's name matches the vendor's legal name	The decision engine's external-signal input
Ramp-signed identity	A vendor already on Ramp proves its bank account with a Ramp-signed statement instead of a call	A new verification method beside the call

Production path (from the decision records): Temporal for orchestration, a daily Merkle root anchored outside Dialback, policy-as-code (Cedar or OPA) if the rules grow, and real phone-menu navigation.

Sources

All pages were accessed on Oct 31, 2026.

Ramp

- Ramp Developer API docs index
- Ramp webhooks guide
- Ramp API changelog

- Ramp Bill Pay API guide
- Ramp rate limiting
- Ramp sandbox access
- Ramp Developer MCP
- Ramp Developer API OpenAPI spec
- Ramp agent tools reference
- Ramp Help Center: Hold payment for bills and vendors
- Ramp Help Center: Bill Pay payment release
- Ramp Help Center: Bill Pay fraud

Voice and telephony

- Twilio ConversationRelay TwiML reference
- Twilio ConversationRelay WebSocket messages
- Twilio: Try out Voice on a trial account
- Twilio US voice pricing
- Retell: Create phone call
- OpenAI: GPT-Live model page
- Vercel: WebSocket support in Vercel Functions

Call protocol, compliance and insurance

- FCC Declaratory Ruling FCC 24-17 on AI-generated voices
- 47 CFR §64.1200
- Justia: Recording phone calls and conversations, 50-state survey
- California Penal Code §632.7
- California Public Utilities Code §2874
- Intact Form M350: Social Engineering Endorsement, Call Back Provision
- Intact Form 71074: Social Engineering Supplemental Application
- Nacha: Business Email Compromise response action plan
- Nacha: Fraud monitoring rules
- J.P. Morgan: 5 ways payments staff can help prevent business email compromise
- FBI IC3: Business email compromise

Trust Ledger

- Claude docs: Structured outputs

- Claude docs: PDF support
- libphonenumber
- RapidFuzz
- NANPA: 555 numbers
- RFC 6761: Special-use domain names
- W3C PROV Primer
- Overture Maps: Places guide

Platform, security and documentation practice

- RFC 8785: JSON Canonicalization Scheme
- microservices.io: Transactional outbox
- Supabase: Custom claims and role-based access control
- Supabase Vault
- Supabase Realtime limits
- Vercel Workflows
- Inngest pricing
- Temporal pricing
- OWASP Top 10 for LLM Applications
- Design Patterns for Securing LLM Agents against Prompt Injections
- Simon Willison on the prompt-injection design patterns
- CaMeL: Defeating Prompt Injections by Design
- The C4 model
- Michael Nygard: Documenting architecture decisions
- Russ Cox: Transparent logs for skeptical clients
- Langfuse pricing

Evals and teamwork

- Binomial proportion confidence interval
- MITRE ATT&CK T1684.001: Impersonation
- promptfoo: Multi-turn red-team strategies
- PyRIT 1.1.0
- Prism
- openapi-typescript

- Turborepo: Structuring a repository
- uv workspaces
- AGENTS.md
- Claude Code: Memory and CLAUDE.md
- Claude Code: Best practices
- GitHub: About code owners